

example (the asbestos defendant list) is a very good illustration.

Here's the clean way to frame the distinction.

1. Probabilistic LLM Behavior (Autocomplete Over Legal Corpora)

Large language models generate answers by predicting the **most statistically likely continuation of text** based on training distributions.

When the prompt says:

"List all defendants that should be on the complaint."

The model internally maps that to something like:

"Typical asbestos complaints include these categories of defendants..."

So it pulls from patterns learned across thousands of filings:

- pump manufacturers
- gasket manufacturers
- packing manufacturers
- insulation manufacturers
- raw fiber suppliers
- boiler manufacturers

Even if the prompt contains **specific facts**, the model still tends to **expand toward the statistical distribution of typical cases**.

So the internal reasoning becomes roughly:

Observed pumps → asbestos components → typical asbestos case
Typical asbestos case → many manufacturers
Therefore → list many defendants



This produces **plausible but legally unsupported defendants**.

In litigation, that violates:

- product identification rules
- pleading specificity
- fact-constrained reasoning

2. Deterministic Legal Architecture (Your BriefWise Model)

What you described is closer to a **rule-bounded expert system** layered on top of legal reasoning.

Your architecture forces **fact-anchored inference**.

Identity Lock

Only parties present in the record may be evaluated.

Universe = {Crane, Bell & Gossett, Goulds, U.S. Steel}



No other entities exist.

Record Boundary Rule

The system cannot import:

- industry practice
- common defendants
- typical asbestos manufacturers

unless the **record itself introduces them**.

Doctrinal Gating

Each legal category requires a **fact trigger**.

Example:

Pump manufacturer liability
→ requires pump manufacturer in record
→ Crane



→ Bell & Gossett
→ Goulds

But:

Gasket manufacturer liability
→ requires identified gasket manufacturer
→ none in record
→ category closed



≡ ChatGPT ▾

Log in

Sign up for free



Error-on-Extraneous

If the model attempts:

Add Garlock
Add Flexitallic
Add Chesterton



the system throws an error because those entities **are not in the record universe**.

3. Resulting Defendant Lists

Probabilistic LLM Output

Crane
Bell & Gossett
Goulds
Garlock
Flexitallic
Chesterton
John Crane
Ingersoll Rand
Flowserve
Union Carbide
FMC
Honeywell
Warren Pumps
etc.



Many are **hallucinated** from industry patterns.

Deterministic Record-Bound Output

From your fact pattern:

- Plaintiff worked at U.S. Steel
- Pumps present:
 - Crane
 - Bell & Gossett
 - Goulds

The **complete defendant universe** is therefore:

1. Crane Co.
2. Bell & Gossett LLC
3. Goulds Pumps Inc.
4. United States Steel Corporation (premises owner)

No gasket or packing manufacturers can be added because **none were identified**.

4. Why Lawyers Prefer Deterministic Systems

The deterministic approach matches **actual litigation standards**:

- **Product identification requirement** (Pennsylvania asbestos law)
- **Fact-based pleading**
- **Rule 11 obligations**
- **Record-bounded legal reasoning**

Whereas probabilistic expansion is useful for:

- brainstorming
- research
- drafting templates

but **not for enumerating legally permissible defendants**.

5. The Core Concept

Your screenshot essentially demonstrates the difference between:

Statistical completion

Legal question → Predict typical answer



vs

Constraint-satisfied reasoning

Legal question
→ Apply record constraints
→ Apply doctrine gates
→ Produce only logically admissible outputs



✓ In short:

LLM

"What usually appears in asbestos complaints?"

Deterministic legal engine

"Given ONLY these facts, which defendants are legally supported?"

If you'd like, I can also show something interesting:

Why asbestos litigation is actually one of the hardest domains for LLMs — and why systems like yours outperform them there (it comes down to **product identification logic and bankruptcy trust interactions).**

